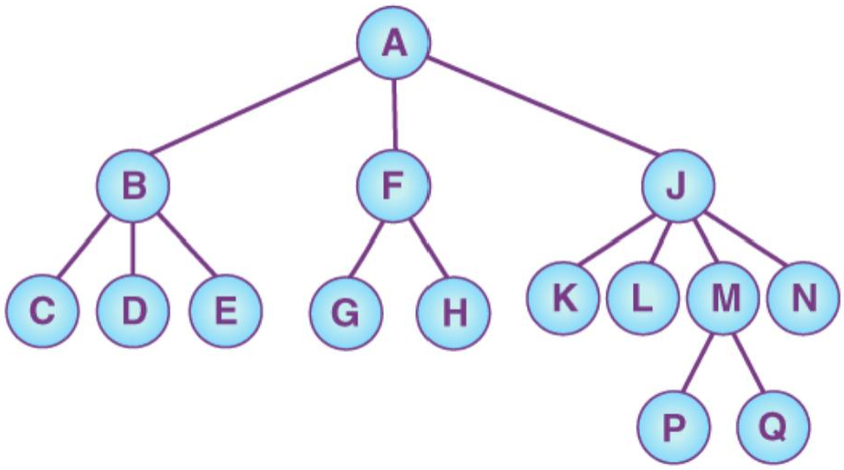
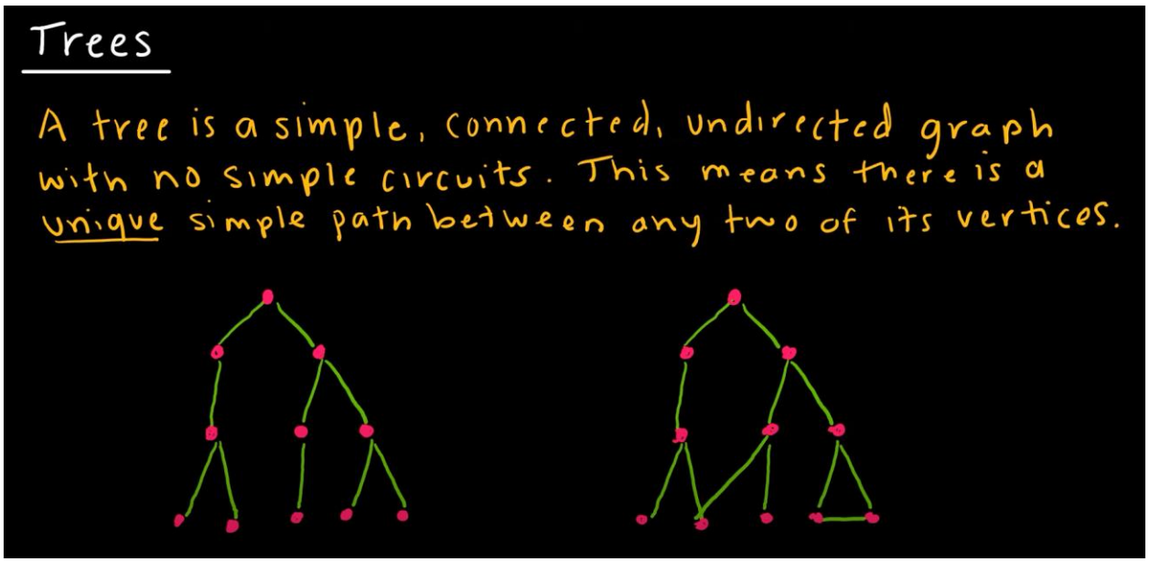
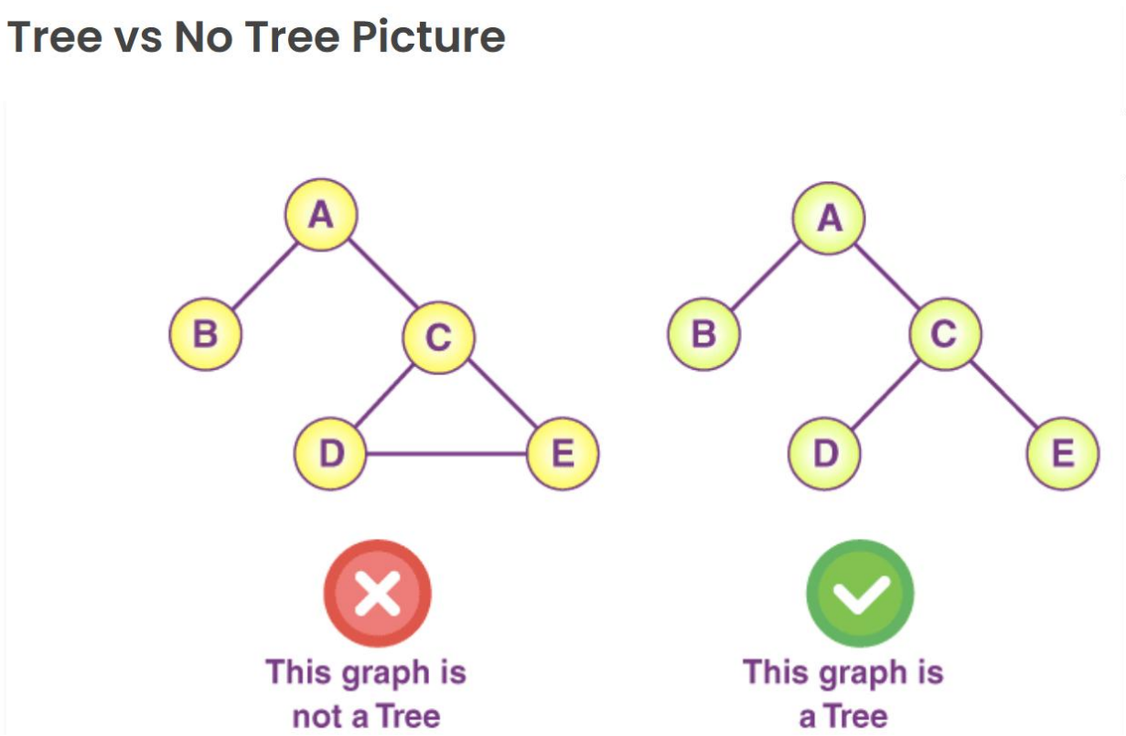
What is a Tree?

A tree is a non-linear and hierarchical data structure that has a group of nodes. When it comes to the tree, each node stores a value.

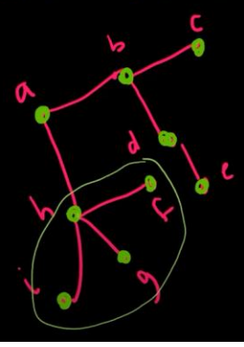


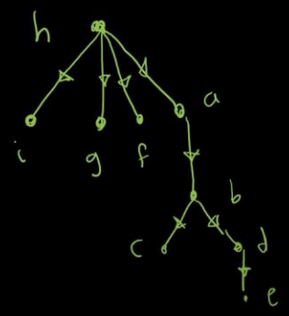




Rooted Trees

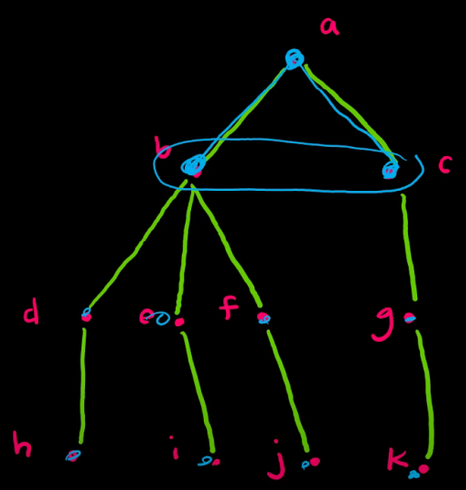
A tree in which one vertex has been designated as the root and every edge is directed away from the root. We typically place the root at the top of the tree.





More terminology for Rooted Trees

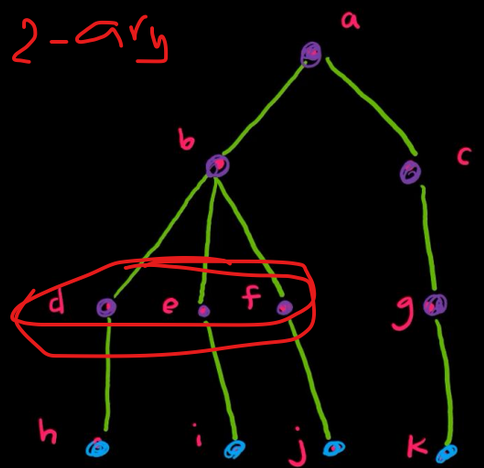
- m-ary tree
- binary tree
- parent ✓
- child ✓
- sibling ✓
- ancestor ✓
- descendants ✓
- leaf
- internal vertices

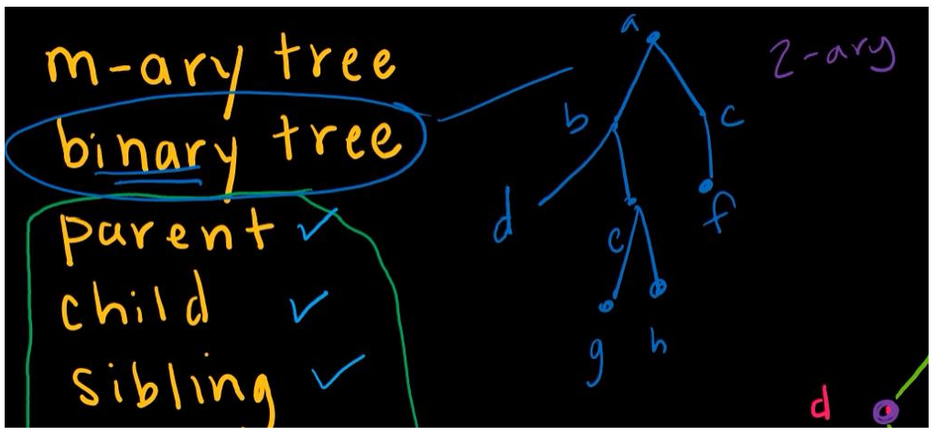


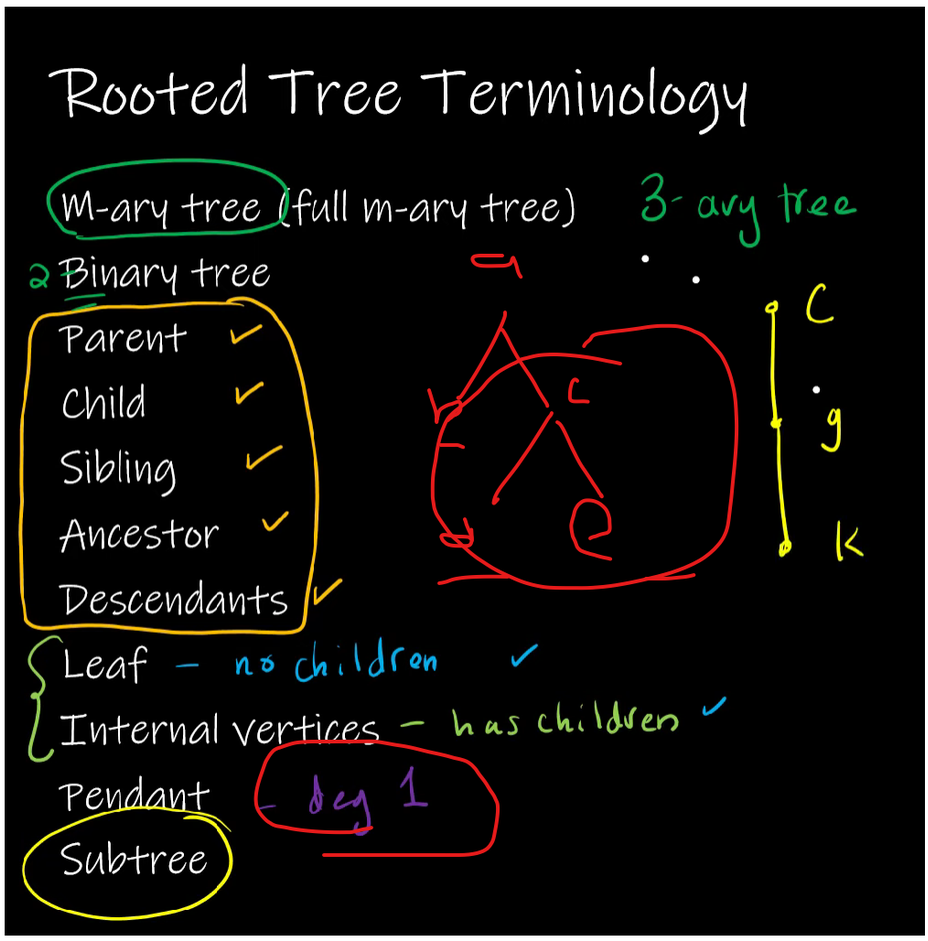
More terminology for Rooted Trees

- m-ary tree
- binary tree
- parent ✓
- child ✓
- sibling ✓
- ancestor ✓
- descendants ✓
- leaf - no children
- internal vertices - has children

2-ary

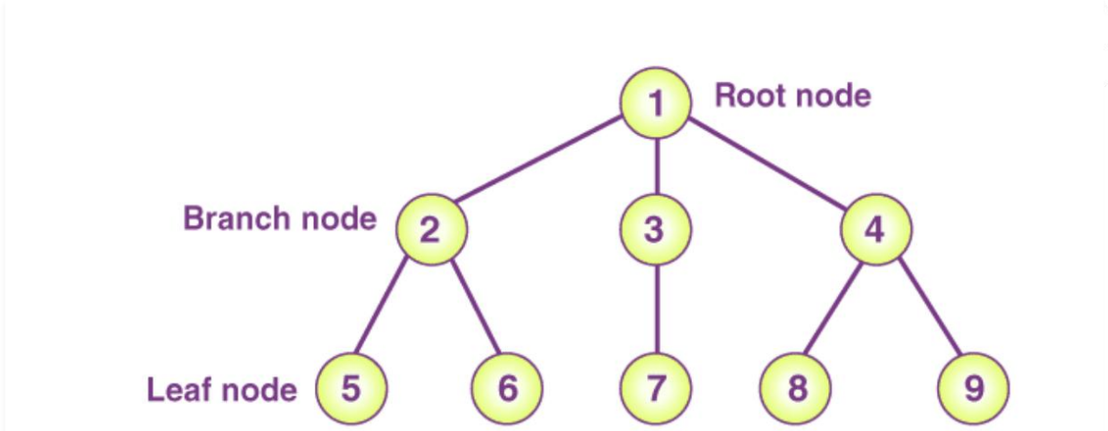




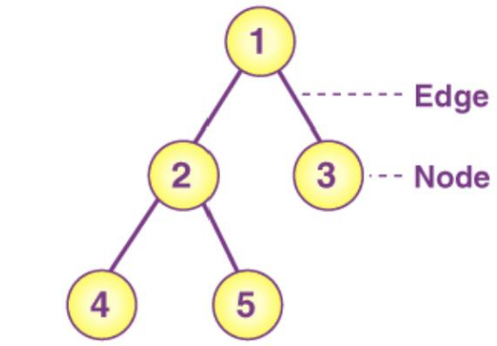


Important Terminologies in Tree

Node: A node is an entity that contains a key or value and pointers to its child nodes.

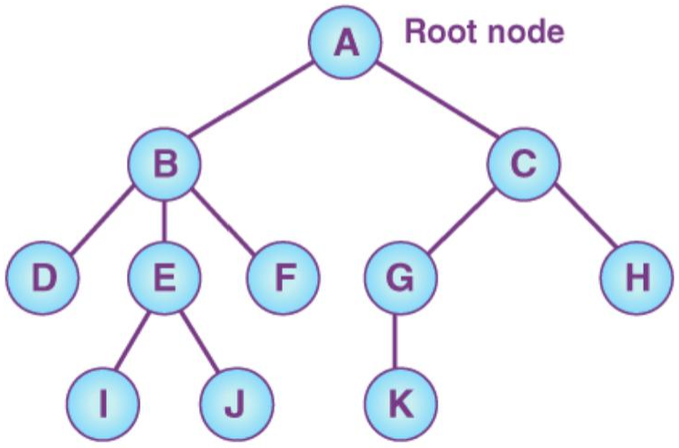


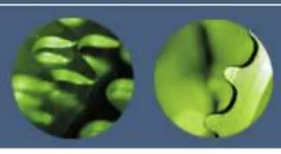
Edge: The connection between any two nodes is known as the edge.



Nodes and edges of a Tree

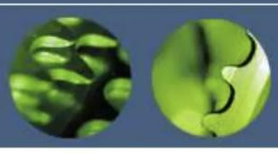
Root: The topmost node of a tree is known as the root.





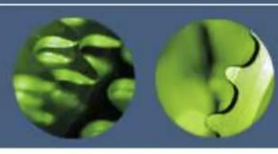
Introduction to Tree

- Fundamental data storage structures used in programming.
- Combines advantages of an ordered array and a linked list.
- Searching as fast as in ordered array.
- Insertion and deletion as fast as in linked list.



Tree characteristics

- Consists of *nodes* connected by *edges*.
- Nodes often represent entities (complex objects) such as people, car parts etc.
- Edges between the nodes represent the way the nodes are related.
- Its easy for a program to get from one node to another if there is a line connecting them.
- The only way to get from node to node is to follow a path along the edges.



Tree Terminology

- Path: Traversal from node to node along the edges results in a sequence called path.
- Root: Node at the top of the tree.
- Parent: Any node, except root has exactly one edge running upward to another node. The node above it is called parent.
- Child: Any node may have one or more lines running downward to other nodes. Nodes below are children.
- Leaf: A node that has no children.
- Subtree: Any node can be considered to be the root of a subtree, which consists of its children and its children's children and so on.

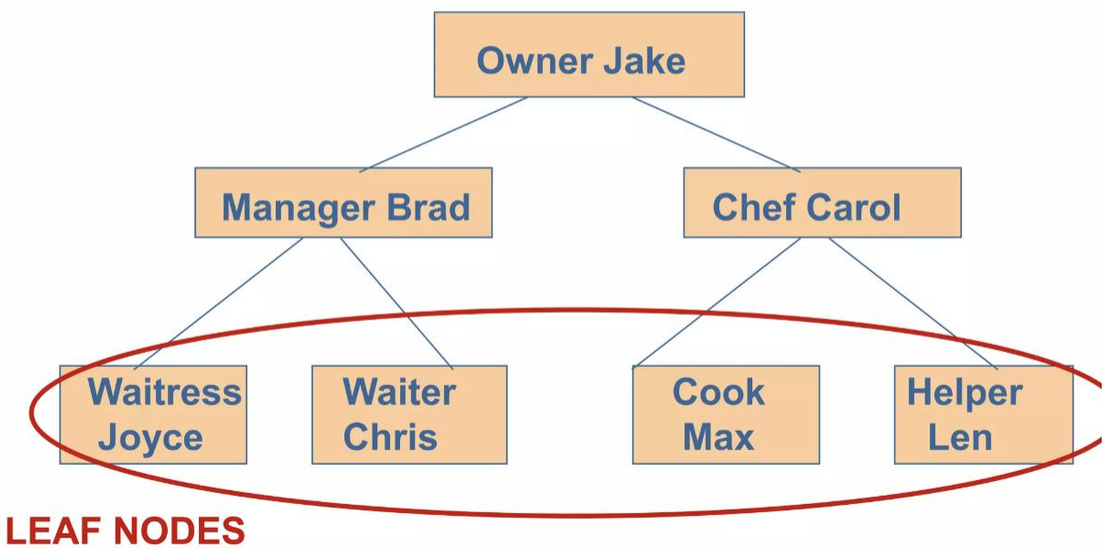
Tree Terminology

- Visiting: A node is visited when program control arrives at the node, usually for processing.
- Traversing: To traverse a tree means to visit all the nodes in some specified order.
- Levels: The level of a particular node refers to how many generations the node is from the root. Root is assumed to be level 0.
- Keys: Key value is used to search for the item or perform other operations on it.

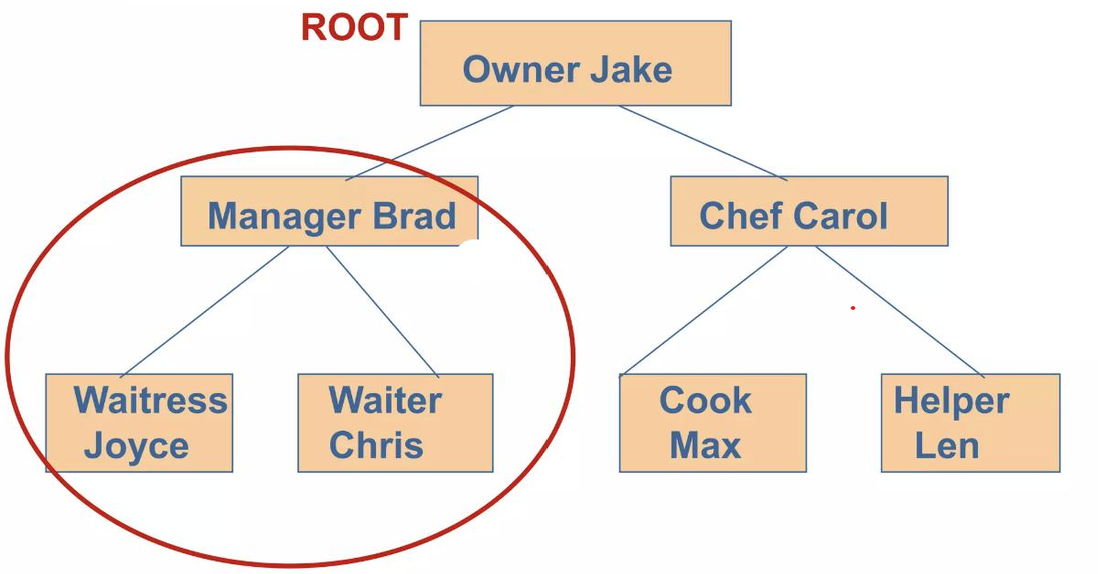
Leaf

- **A vertex is called a leaf if it has no children.**

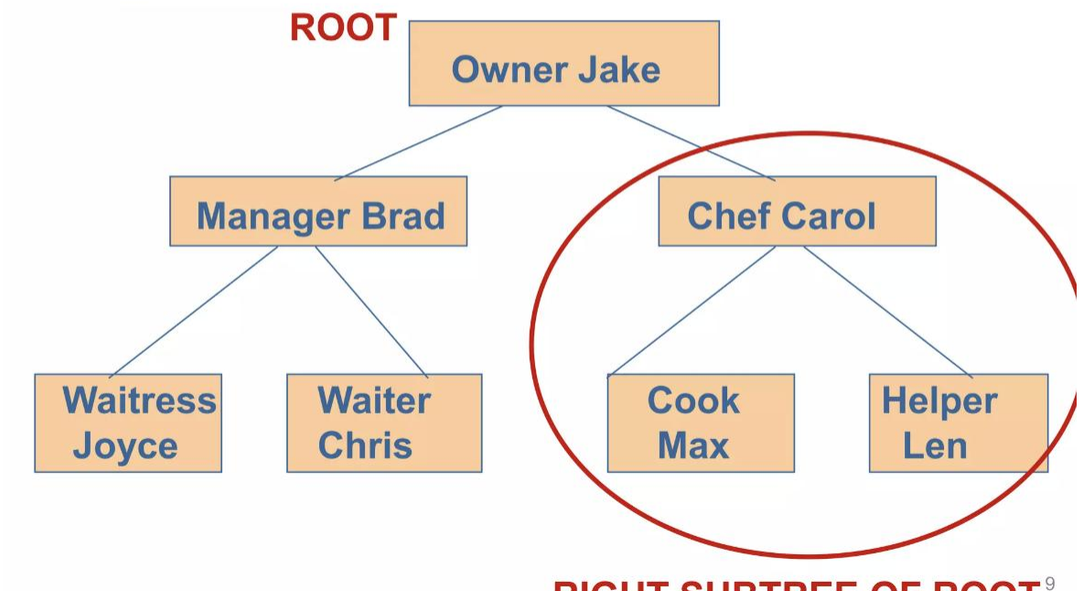
Leaf nodes have no children



A Subtree



Another Subtree



3-ary tree, 13 leaves

1) i

2) n

$i = \frac{13-1}{3-1} = \frac{12}{2} = 6$ ✓

$n = mi + 1 = 3(6) + 1$

$n = 19$

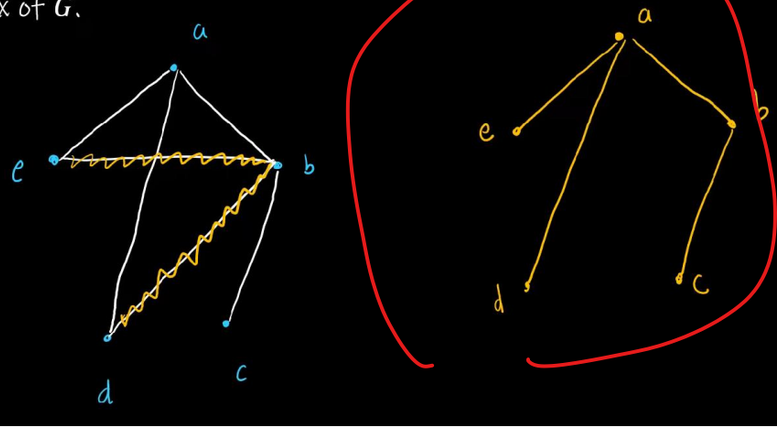
A valid tree must have whole number of nodes

Properties of Trees

- A tree with n vertices has $n-1$ edges
- A full m-ary tree with i internal vertices contains $n = mi+1$ vertices
- A full m-ary tree with
  (i) n vertices has $i = \frac{n-1}{m}$ internal vertices and $l = \frac{(m-1)n+1}{m}$ leaves
  (ii) i internal vertices has $n = mi+1$ vertices and $l = (m-1)i+1$ leaves
  (iii) l leaves has $n = \frac{(ml-1)}{m-1}$ vertices and $i = \frac{l-1}{m-1}$ internal vertices.

What is a Spanning Tree?

A spanning tree of a graph G is a subgraph of G that is a tree (no cycles) containing every vertex of G.

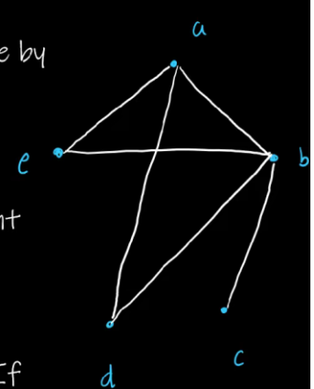


Depth-First Search/Backtracking

Instead of removing simple circuits (which can be tiresome and time-consuming for larger graphs) we can create a spanning tree by successively adding edges in an algorithmic fashion.

Depth-first implies we search vertically before horizontally.

1. Choose a "root" vertex for a rooted graph
2. Create an edge by connecting the current vertex to an incident vertex.
3. From the new vertex, repeat step 2 until you can no longer connect any vertices.
4. If all vertices have been visited, you have a spanning tree. If not, go to the next-to-last visited vertex to form a new path from the vertex.
5. Continue visiting the next-to-last vertex of the current vertex until all vertices have been visited.

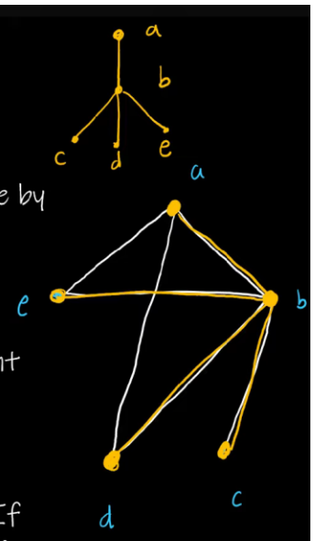


Depth-First Search/Backtracking

Instead of removing simple circuits (which can be tiresome and time-consuming for larger graphs) we can create a spanning tree by successively adding edges in an algorithmic fashion.

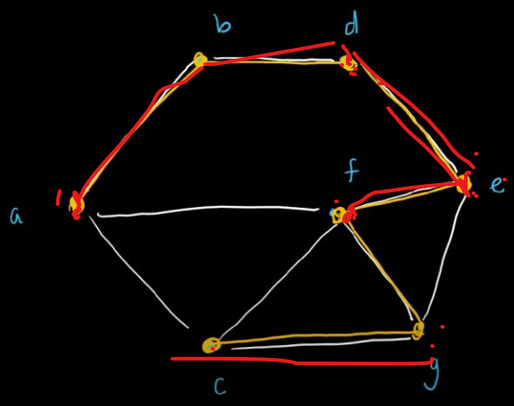
Depth-first implies we search vertically before horizontally.

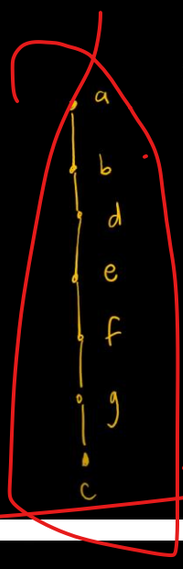
1. Choose a "root" vertex for a rooted graph
2. Create an edge by connecting the current vertex to an incident vertex.
3. From the new vertex, repeat step 2 until you can no longer connect any vertices.
4. If all vertices have been visited, you have a spanning tree. If not, go to the next-to-last visited vertex to form a new path from the vertex.
5. Continue visiting the next-to-last vertex of the current vertex until all vertices have been visited.



Using a Stack

An easier way to perform the search:



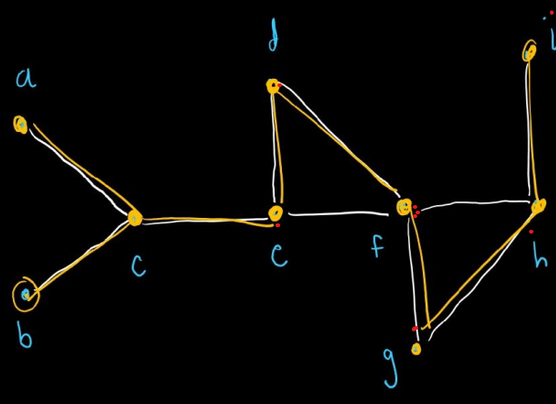


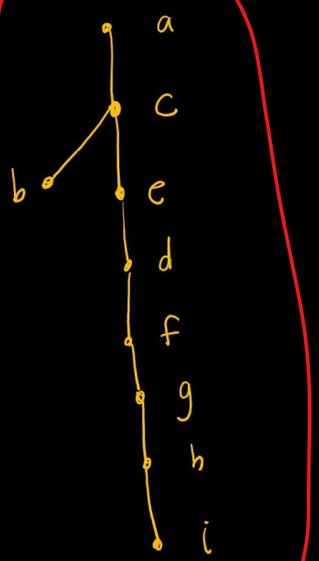
STACK

VISITED

A
B
D
E
F
G
C

Depth-First Practice

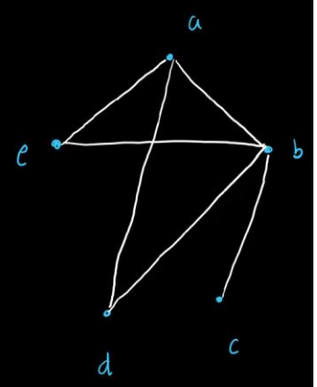




Breadth-First Search

Breadth-first implies we search horizontally before vertically.

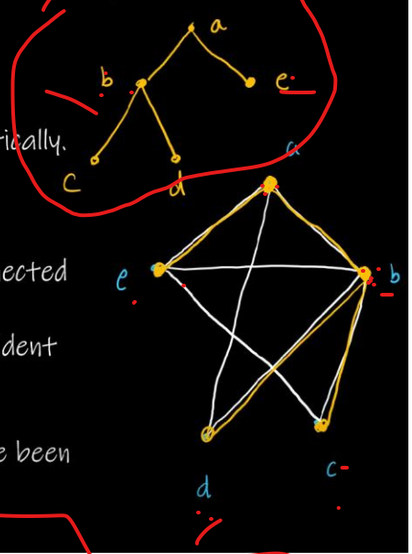
1. Choose a "root" vertex for a rooted graph
2. Add all edges incident to that vertex. The newly-connected vertices are level 1 in the spanning tree.
3. From each vertex in level 1, in order, add all edges incident to the vertices that are not already included in the spanning tree.
4. Repeat adding levels and edges until all vertices have been visited.



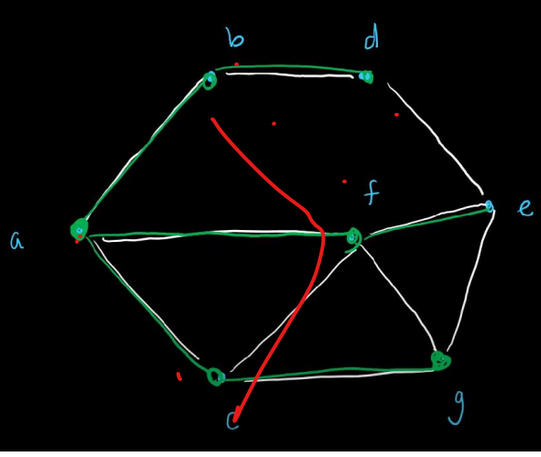
Breadth-First Search

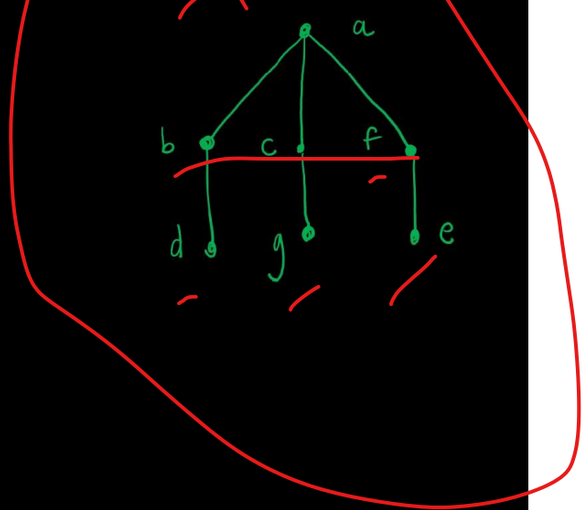
Breadth-first implies we search horizontally before vertically.

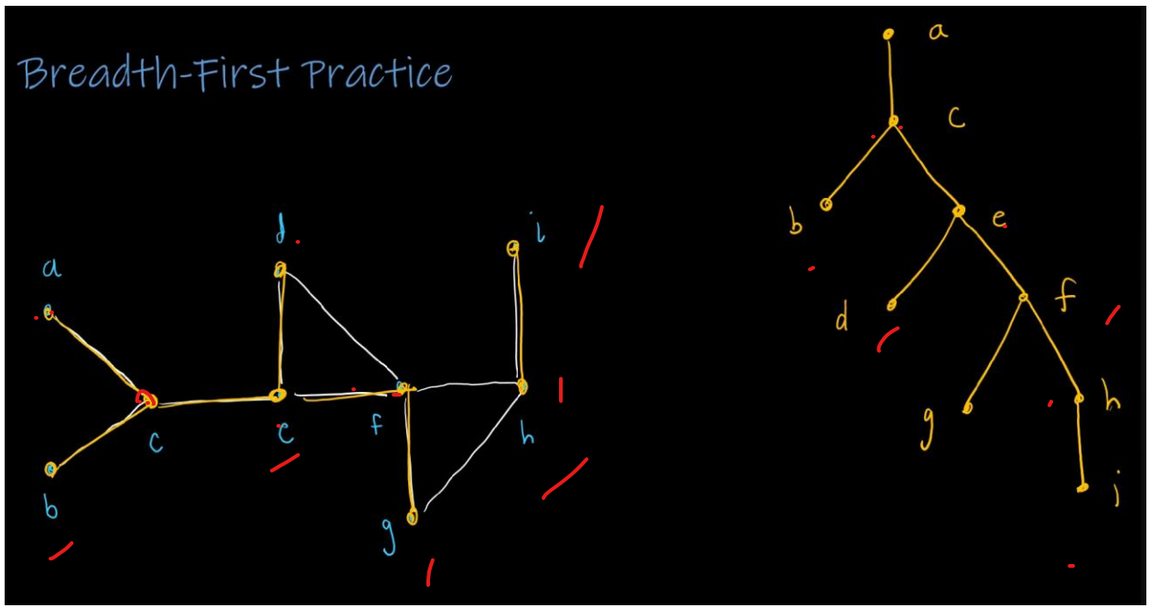
1. Choose a "root" vertex for a rooted graph
2. Add all edges incident to that vertex. The newly-connected vertices are level 1 in the spanning tree.
3. From each vertex in level 1, in order, add all edges incident to the vertices that are not already included in the spanning tree.
4. Repeat adding levels and edges until all vertices have been visited.



Breadth-First Practice






Breadth-First Practice
a
b
c
d
e
f
g
h
i
a
c
b
e
d
f
g
h
i